Claude & GitHub Agents

Best Practices Guide — v2 Enriched Edition

Skills · Hooks · Plugins · MCP · Routing Design · Agent Teams · Anti-Patterns · Token Optimization

April 2026 · Based on Anthropic Official Docs, Community Research & Production Data

60-80%	**27**	**98.4%**
Cost reduction with full optimization	Hook lifecycle events (2026)	Claude Code is deterministic infra
400K+	**35.9K ■**	**7×**
Skills available via SkillKit marketplace	awesome-claude-code GitHub stars	Token cost of Agent Teams vs single

This v2 edition incorporates the **April 2026 Claude Code changelog**, the newly published **Dive-into-Claude-Code architectural analysis** (arXiv 2604.14228), official Anthropic best-practices documentation, production patterns from the awesome-claude-code community (35.9K ■), and a comprehensive anti-pattern catalog drawn from real-world failure modes. New sections cover **Routing Design**, **Agent Teams**, and the **Explore-Plan-Execute pipeline**.

00 Table of Contents

01 Architecture Deep Dive

A 2026 source-level analysis of Claude Code v2.1.88 (~1,900 TypeScript files, ~512K LOC) reveals a counterintuitive truth: **only 1.6% of the codebase is AI decision logic**. The other 98.4% is deterministic infrastructure — permission gates, context management, tool routing, and recovery logic.

■ Architectural Insight
The agent loop is a simple ReAct-pattern while-loop. The real engineering complexity lives in the systems around it: 5 compaction shapers, 9 context sources, 27 hook events, and a 7-layer safety system. Build your agents the same way — keep AI narrow, infrastructure wide.

The 9-Step Pipeline Per Turn

Step	Name	What It Does
1	Settings resolution	Merge user/project/org settings, resolve conflicts
2	State initialization	Session state, active subagents, permission cache
3	Context assembly	Pull from 9 ordered sources (CLAUDE.md, skills, history...)
4-8	5 Compaction shapers	Budget Reduction → Snip → Microcompact → Context Collapse → Auto-Compact
9	Model call + dispatch	Tool requests → Permission gate → Execution → Stop condition check

The 5-Layer Compaction Pipeline

Before every model call, five shapers run sequentially — cheapest first. Each targets a different type of context pressure:

Layer	Name	Triggers When	Cost
1	Budget Reduction	Individual tool output overflows size limit	~0 tokens
2	Snip	Conversation too deep temporally	~0 tokens
3	Microcompact	Cache overhead exceeds threshold	Low
4	Context Collapse	History extremely long	Medium
5	Auto-Compact	Semantic compression needed (last resort)	High

■■ Context Rot — Real Threshold
Quality degrades at 20-40% context full, NOT 80-90% as commonly assumed. Set CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60 to trigger compaction before degradation. The 1M token window on Opus 4.7 is large but does not eliminate this degradation curve.

The 9 Context Sources (Ordered)

Claude Code builds the context window from 9 ordered sources. Understanding this order is critical for debugging unexpected behavior:

- ■ **1.** System prompt (deterministic — always obeyed)
- ■ **2.** Managed settings / org-level policies
- ■ **3.** Project CLAUDE.md (delivered as **user context** — probabilistic compliance, not system prompt!)
- ■ **4.** User-level CLAUDE.md (~/.claude/CLAUDE.md)
- ■ **5.** Active skills (up to 25K shared token budget, most recent first)
- ■ **6.** Subagent system prompts (isolated per subagent)
- ■ **7.** Tool schemas / MCP tool definitions
- ■ **8.** Conversation history (with compaction)
- ■ **9.** Current turn input + tool results

■ Critical: CLAUDE.md is NOT the System Prompt
CLAUDE.md instructions are delivered as user context, giving probabilistic compliance. For deterministic enforcement, use Hooks. For standing domain knowledge, use Skills. CLAUDE.md is for project conventions Claude gets wrong without it — keep it under 80 lines.

Five Workflow Patterns (Anthropic Canonical)

Pattern	Description	Claude Code Implementation
Prompt Chaining	Output of one call feeds next	Sequential skills with context: inline
Routing	Classify input → route to specialist	Subagent descriptions as routing rules
Parallelization	Independent tasks run concurrently	Parallel tool calling or Agent Teams
Orchestrator-Workers	Master delegates to workers	Main agent + subagents (primary pattern)
Evaluator-Optimizer	Output critiqued and improved	Stop hook → evaluate → re-invoke loop

02 Agent Skills — v2 Best Practices

Skills are the primary mechanism for giving Claude domain expertise. The open standard (Dec 2025) means the same SKILL.md works across Claude Code, Claude.ai, the API, Cursor, Gemini CLI, Codex CLI, and Antigravity IDE. The SkillKit marketplace (Apr 2026) now offers 400,000+ skills.

SKILL.md — Full Frontmatter Reference

```
---
name: financial-analysis        # Unique identifier (used for /slash-command)
description: |                  # THE MOST IMPORTANT FIELD — Claude's routing key
  Analyze financial statements and generate Excel reports.
  Use PROACTIVELY when user mentions revenue, P&L, EBITDA, balance sheet,
  cash flow, or asks to 'analyze numbers'. Do NOT use for simple arithmetic.
invocation: auto                # auto | explicit | none
agent: Explore                  # Explore | Plan | general-purpose | custom-name
context: fork                   # fork (isolated) | inline (main context)
allowed-tools: Bash(python3 *), Read, Write
disallowed-tools: Bash(rm *), Bash(curl *)
model: claude-haiku-4-5         # Override model for this skill
permissionMode: plan            # plan | acceptEdits | bypassPermissions
mcpServers:                     # MCP servers available to this skill
  - name: excel-mcp
    url: http://localhost:8100
---
# Financial Analysis Skill
## Dynamic Context Injection (runs before skill loads)
Current date: !`date +%Y-%m-%d`
Python version: !`python3 --version`
## Instructions
You are a CFO-level financial analyst. Follow this exact pipeline:
1. Load file with Read tool
2. Extract: Revenue, COGS, Gross Margin %, EBITDA, FCF
3. Identify anomalies (flag with [ANOMALY] prefix)
4. Generate Excel via xlsx skill
5. Return structured JSON summary
NEVER invent numbers. Confidence score required (0-100).
```

Description Engineering — The Routing Key

The description field is Claude's routing key for auto-invocation. Poor descriptions cause skills to never fire or fire on everything:

Pattern	■ Bad	■ Good

Trigger phrase	'Handles documents'	'Use PROACTIVELY when user asks to create Excel (.xlsx) or analyze spreadsheet data'
Negative boundary	(none)	'Do NOT use for CSV-only tasks, simple text tables, or Google Sheets'
Scope precision	'Financial stuff'	'Income statements, balance sheets, cash flow — NOT general math or budgets'
Action verbs	'For reports'	'Generates, analyzes, extracts, transforms, validates'
Negative examples	(none)	'Examples that should NOT trigger: word count, sorting a list, unit conversion'

SkillTool vs AgentTool — Critical Distinction

Not all skill invocations are equal. The context field determines which mechanism is used:

Field	context: inline (SkillTool)	context: fork (AgentTool)
Effect	Injects into current context window	Spawns isolated context window
Token impact	Consumes from main budget	Separate budget, returns summary only
Use for	Standing instructions, quick lookups	Heavy research, file exploration
Compaction	Shared 25K skill budget	Completely isolated
Can modify files?	Yes (inherits permissions)	Only with allowed-tools: Write

Pre-built Skills (Updated Apr 2026)

Skill ID	Purpose	API Beta	Installs
pptx	Professional PowerPoint presentations	skills-2025-10-02	–
xlsx	Excel with formulas/charts/pivot tables	skills-2025-10-02	–
docx	Word documents with rich formatting	skills-2025-10-02	–
pdf	PDF analysis and extraction	skills-2025-10-02	–
claude-api	Up-to-date API reference, SDK docs	Bundled	–
frontend-design	Distinctive UI avoiding 'AI slop' aesthetics	Bundled	277K+

03 Routing Design & Subagents

Routing is the highest-leverage design decision in any multi-agent system. Claude Code routes via subagent descriptions, model selection per agent, and the Explore-Plan-Execute pipeline. Getting this right prevents the two most common failures: context bloat and wrong model for the task.

The Explore-Plan-Execute Pipeline (Official Pattern)

The canonical routing pattern from Anthropic's internal teams and the official best-practices docs:

```
# CLAUDE.md — Agent Delegation Rules

## Routing Policy

### Explore-Plan-Execute pipeline
For ANY task touching >3 files or involving refactoring:
1. Invoke @explore subagent to map relevant files (read-only, Haiku)
2. Feed output to @plan subagent with specific scope
3. Present plan to user for approval
4. Only then invoke @execute with the approved plan as context

### Direct execution (skip pipeline)
- Quick targeted fix: 1 file, <20 lines
- File already open in conversation context
- Tasks needing frequent back-and-forth

### Domain routing (parallel)
For features spanning multiple domains, spawn parallel agents:
- Frontend (React/CSS): @frontend-agent (Haiku)
- Backend (Node/DB): @backend-agent (Sonnet)
- Security review: @security-agent (Opus)
```

Built-in Subagents (2026)

Agent	Default Model	Tools	Auto-invoked When
Explore	Haiku 4.5	Read, Grep, Glob (read-only)	Search/understand codebase without changes
Plan	Haiku 4.5	Read, Grep, Glob (read-only)	Plan mode — codebase research before strategy
General-purpose	Sonnet 4.6	Full tool access	Complex multi-step tasks needing both exploration and modification

Custom Subagent Design

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: |
  Specialized security code reviewer. Use PROACTIVELY on any changes
```

```
  to auth/, payments/, api/, or middleware/. Checks for OWASP Top 10,
  credential exposure, and injection vulnerabilities.
  Do NOT use for: linting, formatting, test writing.
model: claude-opus-4-6          # Best model for security
context: fork                   # Isolated — returns summary only
allowed-tools: Read, Grep, Glob # Read-only — no modifications
permissionMode: plan            # Show intent before acting
---
You are a senior AppSec engineer. Review changed files for:
- SQL injection, XSS, CSRF, SSRF vulnerabilities
- Hardcoded secrets / credentials in code
- Missing authentication/authorization checks
- Insecure deserialization patterns
- Dependency with known CVEs
SEVERITY LEVELS: [CRITICAL] [HIGH] [MEDIUM] [LOW] [INFO]
Return: structured JSON with findings array + risk_score (0-100).
```

Agent Teams — Experimental Feature (2026)

Agent Teams removes the subagent communication bottleneck. Subagents run within a single session and can only report results back. Agent Teams lets teammates message each other, claim tasks from a shared list, and coordinate directly.

```
# Enable Agent Teams (experimental)
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
# Or enable forked subagents on external builds
export CLAUDE_CODE_FORK_SUBAGENT=1
# Prompt pattern for Agent Teams
# 'Create a team: one for API layer, one for DB migrations, one for test coverage.
#  Coordinate through shared task list.'
```

Feature	Subagents	Agent Teams
Communication	Report to main only	Direct teammate-to-teammate messaging
Context	Single session	Each teammate: own context window
Token cost	~1× single session	~3-4× single session
Time savings	Parallel execution	Parallel + coordinated execution
Best for	Independent parallel tasks	Tasks needing active collaboration
Stability	Production-ready	Experimental (2026)

■■ Agent Teams Cost Warning

Agent Teams use roughly 3-4× the tokens of a single session doing the same work sequentially. Plan mode agent teams cost ~7× tokens. Use Agent Teams only when active coordination between specialists is required — not just parallelism.

Model Selection Routing Guide

Task Type	Model	Reason
Architecture decisions, security review, final review	Opus 4.7	Complex reasoning, low frequency
Feature implementation, debugging, code generation	Sonnet 4.6	Best code quality/cost ratio
Explore subagent, file search, codebase mapping	Haiku 4.5	Fast, cheap, read-only tasks
Linting, formatting, simple transforms	Haiku 4.5	Near-instant, sub-cent per call
Hook evaluation, prompt classification	Haiku 4.5	Semantic classification at minimal cost
Agent Teams — leaf workers	Haiku 4.5	Never use Opus for leaf nodes

■ The /agents Redesign (Apr 2026)

The /agents command now has a tabbed layout: a Running tab shows live subagents, the Library tab adds 'Run agent' and 'View running instance' actions. Use /agents to monitor multi-agent sessions in real time.

04 Hooks — All 27 Events (2026)

Hooks are the deterministic enforcement layer. The April 2026 release expanded hooks significantly: 27 events across 5 categories, 4 execution types, PostToolUseFailure now included, and duration_ms in PostToolUse inputs.

27 Hook Events — Categorized

Category	Event	Can Block?	New in 2026?
User Input	UserPromptSubmit	Yes	–
Tool Lifecycle	PreToolUse	Yes ■	–
Tool Lifecycle	PostToolUse	No	duration_ms added
Tool Lifecycle	PostToolUseFailure	No	■ NEW
Agent Control	Stop	No	–
Agent Control	SubagentStop	No	–
Agent Control	AgentStart	No	–
Agent Control	AgentStop	No	–
Context	PreCompact	No	–
Context	PostCompact	No	–
Notification	Notification	No	–
Monitoring	StatusLine	N/A (display only)	–

4 Handler Types

Type	How It Works	Best For	Error on Failure
command	Shell script via stdin/stdout, exit codes	Local security gates, formatting	Blocks Claude (non-async)
http	POST to web server, JSON body+response	Team-wide policy servers, remote validation	Non-blocking (2xx required to block)
prompt	Sends to Claude model for semantic eval	AI-based code review, quality gates	Depends on fast model response
agent	Spawns subagent with Read/Grep/Glob	Deep verification, complex analysis	Non-blocking

Async Hooks (Jan 2026)

```
# Run in background without blocking Claude
```

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "node notify-slack.mjs",
        "async": true,       // Fire-and-forget
        "timeout": 30        // Still has timeout
      }]
    }]
  }
}
```

PostToolUseFailure — New in 2026

The new PostToolUseFailure event fires when a tool execution fails. It now also receives duration_ms in its input payload, enabling performance-aware error handling:

```
#!/usr/bin/env python3
# PostToolUseFailure hook — log failures + alert on slow timeouts
import json, sys, os
hook = json.load(sys.stdin)
tool = hook.get('tool_name', 'unknown')
error = hook.get('error', '')
duration_ms = hook.get('duration_ms', 0)  # NEW in 2026
# Alert on slow failures (likely timeouts, not logic errors)
if duration_ms > 5000:
    print(f'SLOW_FAILURE: {tool} failed after {duration_ms}ms: {error}',
          file=sys.stderr)
# Log all failures for debugging
with open('/tmp/claude-hook-failures.log', 'a') as f:
    f.write(json.dumps({'tool': tool, 'error': error, 'ms': duration_ms}) + '\n')
```

Security Gate — Bash Hardening (Apr 2026)

The April 2026 changelog patched multiple Bash permission bypasses. Your PreToolUse hook must now handle compound commands, env-var prefixes, redirects, and piped cd segments:

```
#!/usr/bin/env bash
# secure_bash_gate.sh — hardened after Apr 2026 CVEs
INPUT=$(cat)
CMD=$(echo $INPUT | python3 -c 'import sys,json; print(json.load(sys.stdin)["tool_input"].get("command","")))')
# Block dangerous patterns (expanded post-CVE)
DANGEROUS=('rm -rf' 'curl | bash' 'wget | sh' '/dev/tcp' 'base64 -d' 'eval' 'exec'
           '&&' '||'  # Compound commands — now caught
```

```
            'ANTHROPIC_API_KEY='  # Env-var prefix bypass — now caught
          )
  for pattern in "${DANGEROUS[@]}"; do
    if echo "$CMD" | grep -qF "$pattern"; then
      echo '{"decision":"block","reason":"Blocked by security policy"}'
      exit 0
    fi
  done
  echo '{"decision":"approve"}'
```

05 MCP & Plugins

MCP (Model Context Protocol) tools load their definitions directly into context. The GitHub MCP server alone carries 35 tools (~26K tokens of definitions). At scale, MCP tool definitions overload the context window — this is 'the 35-tool problem'.

The 35-Tool Problem & ToolSearch

Anthropic's solution (2026): the **ToolSearch tool**. Instead of loading all tool definitions upfront, Claude can discover tools on demand — reducing initial context overhead dramatically:

```
# Enable ToolSearch in API calls
response = client.messages.create(
    model='claude-sonnet-4-6',
    extra_headers={'anthropic-beta': 'token-efficient-tools-2025-02-19'},
    tools=[
        {'type': 'tool_search_20250515', 'name': 'tool_search'},  # Discovery tool
        # Only add tool definitions Claude actually needs
        {'name': 'get_file', 'description': 'Read a file...', 'input_schema': {...}},
    ],
    messages=[{'role': 'user', 'content': user_message}]
)
# ToolSearch ranking fix (Apr 2026): pasted MCP tool names now surface
# the actual tool instead of description-matching siblings
```

MCP Tool Design Rules

Rule	Description	Impact if Violated
Self-contained	Complete, unambiguous description. Don't rely on Claude's world knowledge	Wrong tool selected
Non-overlapping	Never two tools handling the same input	Non-deterministic selection
Explicit params	Clear types, constraints, examples. Avoid optional params	Malformed calls
Negative examples	Include what NOT to use the tool for (critical boundary definition)	Over-triggering
PROACTIVELY signal	'Use PROACTIVELY when...' prefix for auto-selection	Under-triggering
Limit total tools	Target <20 tools per server (Manus, Claude Code use ~12-20)	Context bloat

Plugin Manifest — 10 Component Types

Claude Code plugins (2026) accept 10 component types in a single manifest:

- commands
- agents
- skills
- hooks
- mcp_servers
- lsp_servers
- output_styles
- channels
- settings
- user_config

Google gws — New MCP Server (Mar 2026)

Google Workspace MCP (gws)
Released March 2026, hit 4,900 GitHub stars in 3 days. One command gives your agent full access to Drive, Gmail, Calendar, and Sheets via a built-in MCP server. Install: npm install -g @googleworkspace/cli && gws mcp -s drive,gmail,calendar,sheets

Concurrent MCP Startup (Apr 2026)

Subagent and SDK MCP server reconfiguration now connects servers in parallel instead of serially. Faster startup when both local and claude.ai MCP servers are configured (concurrent connect now default). Resources/templates/list is deferred to first @-mention — reducing startup latency further.

06 CLAUDE.md & Context Engineering

Context Engineering is the discipline of optimizing token utility within LLM constraints. The CLAUDE.md + .claudeignore pair is the project-level control surface. Memory is file-based — fully inspectable, editable, and version-controllable (no vector DB required).

CLAUDE.md Rules — Official Anthropic Guidance

- **Under 80 lines** — loaded every turn; ruthlessly prune
- **Corrections only** — never repeat what Claude already knows
- **Use imperative language** — 'ALWAYS run tests' not 'prefer to run tests'
- **Architecture, not tutorials** — key dirs, naming conventions, non-obvious decisions
- **Reference skills, not inline** — 'See /financial-analysis for P&L rules'
- **Hooks > CLAUDE.md for enforcement** — CLAUDE.md = probabilistic; hooks = deterministic

Optimal CLAUDE.md Template

```
# Project: MyApp  [KEEP UNDER 80 LINES]

## Architecture (non-obvious only)
- Frontend: Next.js 14 + TypeScript (src/app/)
- Backend: Node.js + Express (src/api/)
- DB: PostgreSQL + Prisma (prisma/schema.prisma)
- Auth: NextAuth.js — use @auth-review subagent before modifying

## Non-Obvious Rules (corrections only)
- API responses MUST use ApiResponse<T> wrapper (src/types/api.ts)
- Never import from 'lodash' directly — use src/utils/lodash.ts re-exports
- .env.local NEVER committed — secrets in Vault, not env vars in code
- Component max 200 lines — split larger ones

## Required Commands
- ALWAYS run: npm test && npm run type-check before committing
- Build: npm run build (check for type errors first)

## Subagent Routing
See agent delegation rules in .claude/agents/ directory.

# [Total: ~25 lines — well under 80 limit]
```

.claudeignore — Context Scoping

```
# .claudeignore — Exclude from Claude's file access
node_modules/       # Never needed
dist/ build/ .next/ # Generated artifacts
*.log *.lock        # Logs and lockfiles
coverage/           # Test coverage reports
.git/               # Git internals
data/raw/           # Large raw data files
```

```
*.csv *.parquet     # Data files (use dedicated data skill instead)
src/generated/      # Auto-generated code
*.d.ts              # TypeScript declarations
__pycache__/ .venv/ # Python artifacts
```

■ Memory Architecture

Claude Code uses file-based memory — no vector DB. This means memory is inspectable, editable, diffable, and version-controllable via git. Use ~/.claude/memory/ for personal persistent context across projects. Use .claude/memory/ for project-specific persistent context shared by the team.

07 Token & Cost Optimization

Combined optimization can reduce monthly API costs by **60-80%** for production agent applications. Average enterprise cost: $13/developer/active day, $150-250/developer/month. 90% of users stay under $30/active day.

60-80%	$13	40-70%
Monthly cost reduction possible	Avg daily cost per developer	Reduction from basic strategies alone

The 6-Tier Optimization Stack

Tier 1: Prompt Caching (Biggest Win)

Cache-aware rate limits (2026): cached tokens no longer count against ITPM limits. This compounds with pricing discounts on cache reads:

```
# Cache system prompt + large knowledge base
response = client.messages.create(
    model='claude-sonnet-4-6',
    extra_headers={'anthropic-beta': 'token-efficient-tools-2025-02-19'},
    system=[{
        'type': 'text',
        'text': SYSTEM_PROMPT + KNOWLEDGE_BASE,
        'cache_control': {'type': 'ephemeral'}  # Cached reads: cheaper + no ITPM
    }],
    messages=conversation_history
)
# Manus team: cache hit rate is their #1 production metric
# Claude Code would be cost-prohibitive without caching
```

Tier 2: Token-Efficient Tool Use

```
# One header = 20-30% output token reduction
extra_headers={'anthropic-beta': 'token-efficient-tools-2025-02-19'}
# Available: Sonnet 4.6, Opus 4.7, Haiku 4.5
# Combined with caching: 60-80% total reduction
```

Tier 3: Model Tiering

Obsessing over Opus vs Sonnet is the 4th or 5th most important optimization. Structural decisions (context, isolation, hooks) matter more:

Task	Model	Why
Architecture, final security review	Opus 4.7	Most capable, infrequent
Feature dev, debugging, code gen	Sonnet 4.6	Best quality/cost ratio
Explore/Plan subagents, search	Haiku 4.5	Fast, cheap, read-only
Hook evaluation, classification	Haiku 4.5	Sub-cent per call

Tier 4: Context Management

- **/clear between tasks** — stale context wastes tokens on every subsequent message
- **CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60** — compact at 60% full, before quality degradation
- **.claudeignore** — exclude node_modules, dist, logs — highest single-file leverage
- **Subagents for exploration** — file reads in subagents return summaries, not raw content
- **Specific file references** — 'Read src/auth/login.ts' beats 'read the auth module'

Tier 5: Preprocessing Hooks

Hooks run outside model context — zero token cost. Highest ROI preprocessing:

- Filter 10,000-line logs → 50 error lines before Claude sees them
- Pre-summarize large JSON API responses to key fields only
- SkillActivationHook: load only relevant skills based on prompt keywords (21 categories)
- Convert binary files to text summaries before Read tool fires

Tier 6: Automation Safeguards

```
# Cap automation — prevent runaway costs in CI/CD
claude --max-turns 15 --timeout-minutes 30 'Run test suite'
# Set workspace spend limits
# Console > Workspaces > Claude Code > Spend Limit
# Environment variables for budget enforcement
export CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60
export ANTHROPIC_BUDGET_TOKEN=your-budget-token
```

Agent Teams Cost Math

Configuration	Token Multiplier	When Worth It
Single session	1×	Always baseline
Subagents (parallel)	~1× (isolated context)	Independent tasks
Agent Teams (3 members)	~3-4×	Active coordination needed
Agent Teams (plan mode)	~7×	Avoid unless critical

08 GitHub Actions & CI/CD

The April 2026 Claude Code update brought major CI/CD improvements: --from-pr now accepts GitLab merge-request, Bitbucket pull-request, and GitHub Enterprise PR URLs. OpenTelemetry tracing now honors privacy flags.

Multi-Platform PR Integration (Apr 2026)

```
# .github/workflows/claude-review.yml
name: Claude AI Code Review
on:
  pull_request:
    types: [opened, synchronize, ready_for_review]
jobs:
  review:
    if: "!contains(github.event.pull_request.labels.*.name, 'skip-review')"
    runs-on: ubuntu-latest
    concurrency:
      group: claude-review-${{ github.event.pull_request.number }}
      cancel-in-progress: true
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          model: claude-sonnet-4-6   # Pin — never use 'latest'
          max_turns: 10
          timeout_minutes: 20
          # Works for: GitHub, GitHub Enterprise, GitLab MR, Bitbucket PR (Apr 2026)
          from_pr: ${{ github.event.pull_request.html_url }}
          prompt: |
            Review for: security vulnerabilities, breaking API changes,
            missing tests, performance regressions. Be specific with line refs.
```

OpenTelemetry Tracing (Privacy-Aware)

```
# .env — Privacy-first tracing (Apr 2026)
OTEL_LOG_USER_PROMPTS=false     # Opt-in only (default: off)
OTEL_LOG_TOOL_DETAILS=false     # Opt-in only
OTEL_LOG_TOOL_CONTENT=false     # Opt-in only
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
```

CI/CD Pattern Matrix

Pattern	Model	Trigger	Max Turns
PR Security Scan	Sonnet 4.6	on: push to any branch	8
Issue Triage & Label	Haiku 4.5	on: issues opened	3
PR Description Generator	Haiku 4.5	on: PR opened (no body)	5
Architecture Review	Opus 4.7	on: PR to main only	15
Dependency Risk Analysis	Sonnet 4.6	on: dependency update PRs	8
Release Notes Generator	Sonnet 4.6	on: push to release/v*	10
Test Failure Analysis	Haiku 4.5	on: CI failure	5

■ CI Cost Tip

Add 'skip-review' label support to skip Claude review on docs-only PRs. Use concurrency groups to cancel in-progress reviews when new commits arrive. Together these reduce CI costs 25-35% on active repos.

09 Security — CVEs & Hardening

Security in Claude Code operates at 7 layers. The April 2026 changelog patched several critical permission bypass vulnerabilities in Bash handling. This chapter covers all known CVEs, the MCP rug pull attack, and hardening patterns.

CVE Registry — Full List (Apr 2026)

CVE	CVSS	Description	Fixed In
CVE-2025-59536	8.7	Hook/MCP init runs before trust dialog — code executes before consent	v1.0.111
CVE-2025-55284	7.1	API key exfiltration via DNS (prompt injection abuses dig/nslookup)	v1.0.4
CVE-2025-52882	8.8	VS Code WebSocket auth bypass — malicious sites can connect (RCE)	VS Code ext v1.0.24+
CVE-2026-21852	5.3	API key exfiltration via malicious ANTHROPIC_BASE_URL	v1.1.x
CVE-2026-33068	7.7	Workspace trust dialog bypass via repo-controlled settings files	v2.1.53
Bash CVE (Apr 2026)	TBD	Backslash-escaped flag auto-allowed as read-only → arbitrary code exec	v2.1.88+
Compound Bash	TBD	Compound commands (&&, ||) bypassed safety prompts	v2.1.88+
POSIX which CVE	TBD	Command injection in POSIX which fallback for LSP binary detection	Patched

■ Critical Rule

NEVER use --dangerously-skip-permissions in directories you don't fully control. CVE-2025-59536 shows that hooks/MCP can execute before the trust dialog — meaning code runs before you can consent. Fixed in v1.0.111 but the flag itself remains dangerous with untrusted repository content.

MCP Rug Pull Attack Pattern

A malicious MCP server can initially present safe tools, earn trust/caching, then change tool behavior mid-session. Defense:

- ■ Pin MCP server URLs to specific commit hashes, not branch names
- ■ Audit all MCP servers before connecting — review source code
- ■ Use PreToolUse hooks to log all tool invocations for audit trail
- ■ Restrict MCP servers to minimum required tools via allowedTools
- ■ Never connect to MCP servers from untrusted repositories

7-Layer Defense Architecture

Layer	Mechanism	Failure Mode

1 - Trust Model	Graduated permissions (sandbox → plan → acceptEdits → bypass)	All share same performance constraints
2 - Permission Gate	PreToolUse hook blocks dangerous actions	50+ subcommands bypass security analysis
3 - Allowlist	Explicit tool permission lists	Over-broad allowlists defeat purpose
4 - Sandboxing	Isolated execution environments	dangerouslySkipPermissions disables all
5 - Hooks	Deterministic enforcement scripts	Async hooks don't block (by design)
6 - Audit Logging	PostToolUse + PostToolUseFailure logging	Logs don't prevent, only detect
7 - Human Review	Human-in-the-loop for destructive ops	27% of tasks attempted only with AI

10 Anti-Patterns Catalog

These are documented failure modes drawn from Anthropic's official best-practices docs, the Dive-into-Claude-Code architectural analysis, and community research across thousands of production deployments.

Skills Anti-Patterns

✗ Anti-Pattern #1: Kitchen Sink CLAUDE.md

Problem: CLAUDE.md over 80 lines, includes things Claude already knows, tutorials instead of corrections. Important rules get lost in the noise — Claude ignores half.

✓ Fix: Ruthlessly prune to <80 lines. If Claude does it correctly without the instruction, delete it or convert to a hook.

✗ Anti-Pattern #2: Description-less Skills

Problem: Skills with vague or absent descriptions: 'Handles documents', 'For financial stuff'. Claude never auto-loads them, or loads them on everything.

✓ Fix: Write descriptions as routing rules with PROACTIVELY signals, negative examples, and explicit trigger phrases.

✗ Anti-Pattern #3: Overlapping Skills

Problem: Two skills that could handle the same input (e.g., 'data-analysis' and 'csv-processor'). Claude picks arbitrarily — non-deterministic behavior.

✓ Fix: Ensure skills are self-contained and non-overlapping. Every skill must justify its existence independently.

✗ Anti-Pattern #4: Inline Everything

Problem: All skill knowledge embedded in CLAUDE.md or as inline system prompt context instead of loading on demand. Wastes tokens every turn even when irrelevant.

✓ Fix: Move domain knowledge to context: fork skills. Load on demand, not on every turn.

Subagent & Routing Anti-Patterns

✗ Anti-Pattern #5: Agent Teams for Simple Parallelism

Problem: Using Agent Teams (3-4× token cost, 7× in plan mode) when subagents would suffice. Teams add coordination overhead and amplify errors.

✓ Fix: Use Agent Teams only when active teammate-to-teammate coordination is required. Use subagents for independent parallel tasks.

✗ Anti-Pattern #6: Raw File Content Returns

Problem: Subagents returning raw file contents to main context instead of summaries. Completely defeats the purpose of context isolation.

✓ Fix: Subagents must return structured summaries. Use 'Return ONLY a structured summary — do not include raw file contents' in system prompt.

✗ Anti-Pattern #7: All-Opus Routing

Problem: Using Claude Opus for every task regardless of complexity. Leaf node agents, explorers, and classifiers running on Opus when Haiku suffices.

✓ Fix: Route by task type: Opus for architecture/security review, Sonnet for implementation, Haiku for search/classify/format.

✗ Anti-Pattern #8: Subagent Nesting

Problem: Attempting to spawn subagents from within subagents (infinite nesting). Claude Code prevents this — subagents cannot spawn other subagents.

✓ Fix: Use the orchestrator-workers pattern with the main agent as orchestrator. Plan subagent handles research in plan mode.

Hooks Anti-Patterns

✗ Anti-Pattern #9: Over-Formatting Hook

Problem: Auto-formatting hooks (prettier/black) running on every Edit. Reported to consume 160K+ tokens in 3 rounds due to context inflation from repeated outputs.

✓ Fix: Run formatters manually between sessions or in PostToolUse with strict output size limits. Consider async hooks to avoid blocking.

✗ Anti-Pattern #10: Complex Slash Commands

Problem: Long list of complex custom slash commands as a substitute for skills. Slash commands are manually invoked; skills auto-load when relevant.

✓ Fix: Convert complex slash commands to skills with invocation: auto. Reserve slash commands for explicit user-triggered actions.

✗ Anti-Pattern #11: Broad allowedTools in Skills

Problem: Skills with allowed-tools: '*' or overly broad permissions. Skills execute code — a malicious skill with broad permissions is a security risk.

✓ Fix: Apply least-privilege to every skill. List only the specific tools needed: allowed-tools: Bash(python3 AnalysisScript.py), Read

Context & Cost Anti-Patterns

✗ Anti-Pattern #12: The Kitchen Sink Session

Problem: One session used for multiple unrelated tasks. Context fills with irrelevant history. Performance degrades silently.

✓ Fix: Use /clear between unrelated tasks. Use /rename before clearing so you can /resume later.

✗ Anti-Pattern #13: Correction Loop

Problem: Claude does something wrong → you correct → still wrong → correct again. Context is now polluted with failed approaches, making the problem worse.

✓ Fix: After two failed corrections: /clear and write a better initial prompt incorporating what you learned.

✗ Anti-Pattern #14: Trust-Then-Verify Gap

Problem: Accepting plausible-looking implementation without verification. AI-generated code has 1.5-2× higher security vulnerability rates than human-written code.

✓ Fix: Always provide verification criteria upfront: tests, linter, screenshot comparison. Claude performs dramatically better when it can verify its own work.

✗ Anti-Pattern #15: Raw Log Reads

Problem: Asking Claude to read a 10,000-line log file to find errors. Consumes tens of thousands of tokens when a grep would suffice.

✓ Fix: Use a PreToolUse hook to preprocess logs before Claude sees them. Filter → 50 relevant lines → massive token reduction.

MCP & Plugin Anti-Patterns

✗ Anti-Pattern #16: 35-Tool MCP Server

Problem: Connecting a single MCP server with 35+ tools (e.g., GitHub MCP server unfiltered). ~26K tokens of tool definitions loaded every turn.

✓ Fix: Use ToolSearch for discovery. Restrict MCP servers to the tools actually needed via the tools parameter.

✗ Anti-Pattern #17: Untrusted MCP Server

Problem: Installing MCP servers from unknown sources without code review. MCP rug pull: server can change tool behavior mid-session after earning trust.

✓ Fix: Only use MCP servers from trusted sources. Pin to commit hashes. Use PreToolUse hooks to audit all invocations.

GitHub Actions Anti-Patterns

✗ Anti-Pattern #18: Unpinned Model Version

Problem: Using model: claude-latest or not pinning the model version in CI/CD. Model updates can change behavior unexpectedly in automated workflows.

✓ Fix: Always pin: model: claude-sonnet-4-6. Update deliberately, not automatically.

✗ Anti-Pattern #19: No Concurrency Group

Problem: Multiple Claude reviews running simultaneously on the same PR when new commits arrive. Wastes tokens and produces conflicting review comments.

✓ Fix: Add concurrency: group: claude-review-${{ github.event.pull_request.number }}, cancel-in-progress: true

✗ Anti-Pattern #20: Artifact Paradox

Problem: Polished AI-generated outputs (code, files) reduce critical human evaluation. Research shows: -5.2pp missing context, -3.7pp fact-checking, -3.1pp reasoning challenge.

✓ Fix: Maintain human review as a required step. Set CLAUDE.md rule: 'I am ultimately responsible for all code in PRs with my name on it.'

11 Latest Additions (Apr 2026)

The awesome-claude-code repo (now 35.9K+ ■, 903 commits) hit issue #1000 on March 12 and posted its April 2026 update on April 6. Submissions continue at ~10+ per week.

New Skills Repos

APR 2026

Repo / Tool	Author	Description
Claude Scientific Skills	K-Dense	Ready-to-use skills for research, science, engineering, analysis, finance, writing. Considered one of best skills repos on GitHub.
Claude Mountaineering Skills	Dmytro Gaivoronsky	Automates mountain route research. Aggregates 10+ sources (Mountaineers.org, PeakBagger, SummitPost) for route beta with weather & avalanche data.
Book Factory	Robert Guss	Pipeline of Skills replicating traditional publishing infrastructure for nonfiction book creation.
Codebase to Course	Zara Zhang	Turns any codebase into an interactive single-page HTML course for non-technical users.
cc-devops-skills	akin-ozer	Detailed DevOps skills: IaC code generation for most major platforms with validations, generators, and CLI tools.
j4rk0r/claude-skills	j4rk0r	3 expert skills: skill-guard (9-layer security auditor), skill-advisor (smart routing), skill-learner (persistent error correction). All scored A+ 120/120.

New Agents & Orchestration

APR 2026

Repo	Description
AgentSys (avifenesh)	Full workflow automation: task-to-production, PR management, code cleanup, drift detection, multi-agent code review. Includes agnix for linting agent configs.
Harness (revfactory)	Meta-skill that designs domain-specific agent teams, defines specialists, and generates their skills. Resources in Korean, English output supported.
awesome-claude-code-toolkit	135 agents across 10 categories, 35 curated skills, 400K+ via SkillKit, 42 commands, 176+ plugins, 20 hooks. Was #1 trending GitHub Feb 2026.
Claude Code Agents (UndeadList)	E2E dev workflow with subagent prompts for solo devs. Parallel auditors, fix cycles with micro-checkpoint protocols, browser QA.

New Dev Tools

APR 2026

Tool	Type	Key Feature
claude-devtools (matt1398)	Desktop app	Session log analysis, turn-based context data, compaction visualization, subagent execution trees, custom notification triggers.
notch-so-good	macOS notch widget	Pixel-art crab lives in Mac notch watching Claude Code. Live timers, color-coded notifications, 13 idle animations, mouse-reactive eyes.

codebase-graph	MCP + npm	42-language tree-sitter AST parsing, FalkorDB knowledge graphs, 0.944 MRR search quality. npm: @anthropic/codegraph
Claudex (Kunwar Shah)	Web browser	Browse Claude Code conversation history across projects. Full-text search, high-level analytics, export options. Completely local, no telemetry.
CodeBurn	CLI dashboard	CLI usage analytics: costs, token consumption, efficiency scoring, weekly reports. Supports Codex and Cursor too.

April 2026 Changelog Highlights

CHANGELOG

- **--from-pr** now accepts GitLab MR, Bitbucket PR, and GitHub Enterprise PR URLs
- **PostToolUseFailure hook** + duration_ms in PostToolUse inputs
- **MCP parallel startup** — servers connect concurrently instead of serially
- **CLAUDE_CODE_FORK_SUBAGENT=1** — enables forked subagents on external builds
- **/agents redesign** — tabbed layout with Running and Library tabs
- **/resume** now offers to summarize stale large sessions before re-reading
- **Bash permission hardening** — compound commands, env-var prefixes, redirects patched
- **Opus 4.7 context fix** — was incorrectly computing against 200K instead of 1M window
- **OTEL privacy flags** — user prompts and tool content now opt-in for tracing
- **Plugin auto-dependency resolution** — claude plugin marketplace add now resolves deps

Academic Resources

RESEARCH

- **Dive-into-Claude-Code** (arXiv 2604.14228) — Source-level analysis of v2.1.88. Key finding: 98.4% deterministic infra, 1.6% AI. Five values → 13 principles → implementation.
- **FlorianBruniaux/claude-code-ultimate-guide** — 41 interactive diagrams, 9-category quiz, configuration decision guide across all 7 config layers.
- **VILA-Lab/Dive-into-Claude-Code** — 27 hook events (not 12), 10-component plugin manifest, 15+ SKILL.md frontmatter fields documented from source.

12 Quick Reference & Checklists

New Project Setup Checklist

1. Create CLAUDE.md (<80 lines, corrections only, architecture overview)
2. Add .claudeignore (node_modules, dist, logs, *.csv, *.lock, .git)
3. Set CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60 in environment
4. Install relevant pre-built skills: pptx, xlsx, docx, pdf, frontend-design
5. Create .claude/agents/ with domain-specific subagent definitions
6. Add PreToolUse security hook (credential protection + Bash hardening)
7. Add PostToolUse formatting hook (pre-commit or prettier — async: true)
8. Add Stop hook for async notifications (async: true)
9. Set up cost monitoring: ccflare or ccxray dashboard
10. Configure workspace spend limits in Claude Console
11. Add token-efficient-tools beta header to all API calls
12. Enable prompt caching on system prompts and large repeated context
13. Pin model versions everywhere — never use 'latest' in production
14. Add SKIP_REVIEW label support to GitHub Actions workflows
15. Add concurrency groups to all Claude CI/CD jobs

Mechanism Decision Guide

'When should I use...'	Answer
CLAUDE.md	Project conventions Claude gets wrong without it. Under 80 lines only.
Skills	Domain knowledge, org workflows, tasks needing supporting files/scripts.
Hooks	Anything that MUST always happen. Deterministic enforcement only.
Subagents	Side tasks that would flood main context with search results or file contents.
Agent Teams	Active coordination between specialists required — NOT just parallelism.
MCP	External services, APIs, databases. Keep total tools <20 per server.
Slash commands	Explicit user-triggered workflows. Not for things Claude should auto-detect.

Key Environment Variables

Variable	Purpose	Recommended Value
CLAUDE_AUTOCOMPACT_PCT_OVERRIDE	Trigger compaction before quality degrades	60
CLAUDE_CODE_FORK_SUBAGENT	Enable forked subagents on external builds	1

CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS	Enable Agent Teams (experimental)	1
CLAUDE_CODE_HIDE_CWD	Hide working dir in startup logo	1 (for security)
OTEL_LOG_USER_PROMPTS	Include user prompts in telemetry	false (default)
ANTHROPIC_BUDGET_TOKEN	Budget enforcement token	your-budget-token

Essential Reference Repos

Repo	Stars	What to Get From It
hesreallyhim/awesome-claude-code	35.9K ■	Curated skills, hooks, agents — start here
rohitg00/awesome-claude-code-toolkit	Rising	135 agents, 400K+ skills via SkillKit
VILA-Lab/Dive-into-Claude-Code	Academic	Source-level architecture analysis
FlorianBruniaux/claude-code-ultimate-guide	Community	41 diagrams, decision trees, quizzes
K-Dense-AI/claude-scientific-skills	New	Research, science, engineering, finance skills
anthropics/claude-code (Releases)	Official	Changelog, CVE fixes, new features

■ Final Summary

The biggest wins are structural, not model selection. Ranking by impact: (1) Context architecture — .claudeignore, skills, compaction tuning. (2) Prompt caching — system prompt + cache-aware ITPM. (3) Hook enforcement — convert best practices to deterministic rules. (4) Model routing — Haiku for leaf nodes, Opus only for critical review. (5) Model selection — last, not first. Developers consistently achieve 60-80% cost reduction with structural changes alone, often in one day of setup.